Système d'exploitation : TP2 (fork, wait et exec)

février 2026 — Pierre Rousselin

Exercice 1 : `exec`

L'appel système `exec` *remplace* le programme d'un processus par un autre programme. Il en existe 6 versions. Nous donnons l'exemple de `execlp`, les 5 autres sont assez semblables (voir `man 3 exec`).

```
#include <unistd.h>
int execlp(const char *chemin, const char *arg0, ..., NULL);
```

Par convention, `arg0` est égal à `chemin` et la liste *doit* se terminer par l'adresse `NULL`. Le `l` de `execlp` signifie *list* et le `p` signifie `PATH` : les répertoires du `PATH` sont utilisés pour la recherche de commande s'il n'y a pas de `/` dans `chemin`.

1. Télécharger et lire le programme `exemple_execlp.c` sur l'ENT. Le compiler et l'exécuter.

2. Le modifier de façon à ce que, au lieu d'afficher le contenu de `/usr`, il fasse une copie de `exemple_execlp.c` nommée `exemple_copie.c` en exécutant le programme `cp`.

3. Écrire un programme qui crée trois processus :

a) Un premier enfant est créé, qui devient `ls` avec comme argument le premier argument du processus (`argv[1]`).

b) Après avoir attendu la terminaison du premier enfant, un second enfant est créé, qui devient `du` (pour *disk usage*) avec les options `-sh` (pour *summarize et human-readable*) et comme argument, encore une fois le premier argument du processus.

c) Enfin, le parent devient `df` (pour *disk free*) avec encore comme argument `argv[1]`.

Voir l'exemple suivant :

```
$ ./du_df_ls du_df_ls.c
du_df_ls.c
4,0K        du_df_ls.c
Sys. de fichiers blocs de 1K  Utilisé Disponible Uti% Monté sur
/dev/sda2          249009152 61561396  187342780  25% /home
```

--- * ---

Correction de l'exercice 1 :

Question 2 : remplace la ligne `execlp` par

```
execlp("cp", "cp", "exemple_execlp.c", "exemple_copie.c", NULL);
```

Question 3 :

```
#include <unistd.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
        if (argc < 2) {
                fprintf(stderr, "Usage: %s PATHNAME\n", argv[0]);
                return 1;
        }
        switch (fork()) {
        case -1:
                perror("fork 1");
                return 1;
        case 0:
                execlp("ls", "ls", argv[1], NULL);
```

```
        default:
                wait(NULL);
                switch (fork()) {
                case -1:
                        perror("fork 2");
                        return 1;
                case 0:
                        execlp("du", "du", "-sh", argv[1], NULL);
                default:
                        wait(NULL);
                        execlp("df", "df", argv[1], NULL);
                }
        }
        return 0; /* Jamais atteint */
}
```

--- * ---

Exercice 2 : `multiexec`

Écrire un programme `multiexec` tel que la ligne de commande

```
$ ./multiexec n cmd arg1 ... argk
```

lance n copies (par fork) de la commande `cmd arg1 arg2 ... argk`. Voir l'exemple suivant.

```
$ ./multiexec 3 ls /usr
bin  etc  games  include  lib  lib64  libexec  local  sbin  share  src  tmp
bin  etc  games  include  lib  lib64  libexec  local  sbin  share  src  tmp
bin  etc  games  include  lib  lib64  libexec  local  sbin  share  src  tmp
```

Vous utiliserez `execvp` (voir `man 3 exec`). Le programme devrait s'arrêter et donner un message d'erreur lorsque le nombre d'arguments n'est pas suffisant.
Question bonus : faire en sorte que le statut de sortie de la commande

```
$ ./multiexec n cmd arg1 ... argk
```

soit celui de la dernière commande `cmd arg1 ...` à avoir terminé si elle s'est terminée normalement ou bien $128 +$ `SIGNUM` si celle-ci a été tuée par un signal de numéro `SIGNUM`. Pour cela voir les macros dans `man 2 wait` et l'exemple suivant :

```
$ ./multiexec 2 ls azerty
ls: ls: impossible d'accéder à 'azerty'impossible d'accéder à 'azerty':
Aucun fichier ou dossier de ce type
: Aucun fichier ou dossier de ce type
$ echo $?
2
$ ls azerty
ls: impossible d'accéder à 'azerty': Aucun fichier ou dossier de ce type
$ echo $?
2
$ ./multiexec 2 sleep 4&
[1] 23173
$ killall sleep
$
[1]+  Termine 143             ./multiexec 2 sleep 4
```

--- * ---

Correction de l'exercice 2 :

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void usage_and_exit(void);

int main(int argc, char *argv[])
{
        int n, status;
        if (argc < 3)
                usage_and_exit();
        n = atoi(argv[1]);

        for (; n > 0; --n) {
                switch (fork()) {
                case -1:
                        perror("fork");
                        return 1;
                case 0:
                        execvp(argv[2], argv + 2);
                        perror("exec");
                        return 127;
                }
        }

        while (wait(&status) != -1)
                ;
        return WIFEXITED(status) ? WEXITSTATUS(status) : 128 + WTERMSIG(status);
}

void usage_and_exit(void)
{
        puts("Usage: lance NUM CMD_NAME [ARG]...");
        exit(1);
}
```

--- * ---

Exercice 3 : `minishell`

1. Écrire un shell minimal, capable d'exécuter des commandes simples sans argument entrées par l'utilisateur. Voici un exemple de déroulement de ce programme (au départ on est dans le shell habituel, ensuite dans le minishell et enfin de nouveau dans le shell habituel) :

```
pierre@noether$ ./minishell
$ ls
a.out  ex2.c  exemple_execlp.c minishell.c   ex1.c  ex3.c  minishell
$ ps
  PID TTY          TIME CMD
 2452 pts/1    00:00:00 bash
 7179 pts/1    00:00:00 minishell
 7185 pts/1    00:00:00 ps
$ <C-d>
pierre@noether$
```

   On pourra utiliser la fonction `fgets` pour lire une ligne d'entrée standard et remplacer le caractère `'\n'` par `'\0'` puis utiliser `execlp`.

2. Ajouter les fonctionnalités suivantes à votre minishell :

- Gérer les arguments des commandes (disons, séparés par des espaces). Vous pourrez pour cela utiliser les fonctions `strtok` (de `<string.h>`) et `execvp`.
- Faire en sorte que si le dernier caractère de la ligne est `&`, la commande est lancée en arrière-plan : le parent n'attend pas qu'elle se termine avant de demander une nouvelle commande à l'utilisateur.
- Ajouter les commandes internes (c'est-à-dire directement prises en charge par votre shell) `pwd` et `cd` pour afficher le répertoire courant et changer de répertoire courant (voir `man 3 getwd` et `man 2 chdir`)
- Si vous avez le temps, vous pouvez ajouter autant de fonctionnalités que vous voulez, par exemple la gestion des variables (`var=truc`) et leur utilisation (`echo $var`), l'utilisation d'autres commandes internes (par exemple pour `echo`) à la place de programmes externes, ...

--- * ---

Correction de l'exercice 3 : Pour le premier `minishell` je n'ai fait aucune gestion des erreurs (par exemple ligne vide).

```
#include <sys/types.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdio.h>
#include <string.h>

#define MAXLIGNE 1024
#define PROMPT "$ "

int main(void)
{
        char ligne[MAXLIGNE];
        printf(PROMPT);
        while (fgets(ligne, MAXLIGNE, stdin)) {
                /* remplacer \n par \0 */
                ligne[strlen(ligne) - 1] = '\0';
                switch(fork()) {
                case -1:
                        perror("fork");
                        return 1;
                case 0:
                        execlp(ligne, ligne, NULL);
                        fprintf(stderr, "Erreur : ne peut exécuter %s\n", ligne);
                        return 1;
                default:
                        wait(NULL);
                        printf(PROMPT);
                }
        }
        printf("\n");
        return 0;
}
```

Le second est un peu plus sophistiqué.

```
#include <sys/types.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdio.h>
```

```c
#include <string.h>

#define MAXLIGNE 1024 /* en comptant \n et \0 */
#define MAXTOKS 20
#define PROMPT "$ "
#define FS " \t\n"
int main(void)
{
	char ligne[MAXLIGNE];
	char *args[MAXTOKS];
	printf(PROMPT);
	while (fgets(ligne, MAXLIGNE, stdin)) {
		int attendre = 1;
		int i = 0;
		args[i] = strtok(ligne, FS);
		while ((args[++i] = strtok(NULL, FS)))
			;
		if (args[i - 1] == NULL) {
			printf(PROMPT);
			continue;
		}
		if (!strcmp(args[i - 1], "&")) {
			if (i == 1) {
				printf(PROMPT);
				continue;
			}
			attendre = 0;
			args[i - 1] = NULL;
		}
		switch (fork()) {
		case -1:
			perror("fork");
			return 1;
		case 0:
			execvp(args[0], args);
			perror("exec");
			fprintf(stderr, "minishell: ne peut exécuter %s\n", ligne);
			return 127;
		default:
			if (attendre)
				wait(NULL);
			printf(PROMPT);
		}
	}
	printf("\n");
	return 0;
}
```

Pour le reste, il n'y a pas encore de correction (si quelqu'un veut se dévouer pour le faire, il ou elle est la bienvenue).

--- * ---